

CLASE 08 — TU PROYECTO ASIGNADO

JOIN y consultas relacionales aplicadas a las relaciones reales de tu dominio

Guía de trabajo del estudiante · Observa ParkFlow 360, replica el método en TU proyecto

Regla de esta guía

ParkFlow 360 NO es tu proyecto. El profesor lo utiliza para demostrar el método. Tu evidencia debe construirse con las tablas, relaciones, reglas y preguntas de negocio de la ficha oficial asignada a tu grupo.

1. Qué debes conseguir en esta clase

En la Clase 07 aprendiste a consultar y modificar datos de una tabla. Ahora debes demostrar que comprendes el carácter relacional de tu base: combinar información de varias tablas siguiendo PK y FK, sin copiar consultas de ParkFlow que no tengan sentido en tu dominio.

- Identificar al menos dos relaciones reales de tu modelo físico.
- Construir INNER JOIN de dos tablas.
- Construir al menos un JOIN encadenado de tres tablas cuando el modelo lo permita.
- Construir un LEFT JOIN que responda una pregunta donde interesen registros sin coincidencia.
- Explicar aliases, ON, WHERE y NULL.
- Predecir qué representa cada fila del resultado.
- Diagnosticar un JOIN incorrecto.
- Guardar y versionar tus consultas como evidencia del proyecto.

2. Qué debes observar mientras el profesor trabaja ParkFlow 360

- Cómo localiza primero la FK antes de escribir JOIN.
- Cómo traduce una pregunta de negocio a un recorrido entre tablas.
- Cómo usa aliases sin perder claridad.
- Cómo predice la cantidad aproximada de filas.
- Por qué INNER JOIN elimina filas sin coincidencia.
- Por qué LEFT JOIN conserva filas y puede producir NULL.
- Cómo distingue ON de WHERE.
- Cómo reconoce duplicados aparentes producidos por relaciones 1:N.
- Cómo detecta un producto cartesiano o un ON incorrecto.
- Cómo guarda consultas útiles en Git.

3. Lo que NO debes copiar

- No copies parking_lot, zone, parking_space, customer, vehicle o reservation si no existen en tu dominio.
- No copies relaciones de ParkFlow sólo porque el SQL ejecuta.
- No inventes una relación que no esté respaldada por una FK de tu modelo.
- No uses DISTINCT para esconder resultados que no comprendes.
- No presentes consultas sin explicar qué pregunta de negocio responden.

4. Archivos y recursos que debes tener abiertos

- Ficha oficial del proyecto asignado.
- DER lógico y diccionario de datos de Clase 04.
- modelo-fisico-v0.1.md y plan-migracion-v1.md.
- V1__init_core.sql de Clase 06.
- V2__seed_core.sql y consultas de Clase 07.
- DataGrip conectado a TU base de datos.
- Repositorio actualizado.

5. Paso 1 — Selecciona relaciones reales de TU proyecto

No empieces escribiendo SQL. Primero dibuja o escribe la relación usando tabla padre, PK, tabla hija y FK.

Relación	Tabla padre	PK padre	Tabla hija	FK hija	Cardinalidad
Ejemplo genérico	cliente	id	vehiculo	cliente_id	1:N
Relación 1 de tu proyecto	_____	_____	_____	_____	_____
Relación 2 de tu proyecto	_____	_____	_____	_____	_____
Relación 3 de tu proyecto	_____	_____	_____	_____	_____

Regla

Si no puedes señalar la FK física que conecta las tablas, todavía no estás listo para escribir el JOIN.

6. Paso 2 — Formula preguntas de negocio

Cada JOIN debe existir porque necesitas responder algo útil. Escribe la pregunta antes del SQL.

Tipo	Ejemplo de forma	Tu pregunta
JOIN 2 tablas	“Mostrar cada X con el nombre de su Y.”	_____
JOIN 3 tablas	“Mostrar X con datos de Y y Z.”	_____
LEFT JOIN	“Mostrar todos los X, incluso si no tienen Y.”	_____
Sin coincidencia	“Mostrar X que todavía no tienen Y.”	_____

7. Paso 3 — INNER JOIN de dos tablas

Plantilla — adapta nombres reales

```
SELECT
  a.columna,
  b.columna
FROM tabla_hija a
INNER JOIN tabla_padre b
  ON b.id = a.padre_id;
```

Después de ejecutarlo, responde por escrito:

- ¿Qué representa una fila?
- ¿Cuál es la PK?

- ¿Cuál es la FK?
- ¿Por qué la condición ON es correcta?
- ¿Qué filas quedarían fuera si no tienen coincidencia?

8. Ejemplos de transferencia por dominio

Proyecto / dominio	Posible relación para JOIN	Pregunta útil
BiblioSmart	prestamo → ejemplar → libro	¿Qué libro corresponde a cada préstamo?
CargoTrack	envio → ruta / historial_estado	¿En qué ruta y estado se encuentra cada envío?
SalonPro	cita → profesional → servicio	¿Qué profesional y servicio corresponden a cada cita?
EquipRent	reserva_detalle → unidad_equipo → equipo	¿Qué unidad y tipo de equipo contiene cada reserva?
FarmaStock 360	venta_detalle → producto / lote	¿Qué producto y lote participan en una venta?
AutoCare 360	orden_servicio → vehiculo → cliente	¿A qué vehículo y propietario pertenece cada orden?

Estos son ejemplos de razonamiento. Usa únicamente relaciones que realmente existan en tu ficha y en tu modelo implementado.

9. Paso 4 — JOIN encadenado de tres tablas

Plantilla conceptual

```
SELECT
    a.columna,
    b.columna,
    c.columna
FROM tabla_a a
JOIN tabla_b b ON b.id = a.b_id
JOIN tabla_c c ON c.id = b.c_id;
```

Antes de ejecutar

Dibuja el recorrido $A \rightarrow B \rightarrow C$. Si estás “saltando” una tabla necesaria, probablemente tu condición sea incorrecta.

10. Paso 5 — LEFT JOIN

Usa LEFT JOIN cuando tu pregunta necesite conservar todas las filas de la tabla izquierda aunque todavía no exista información relacionada.

Plantilla

```
SELECT
    p.id,
    p.nombre,
    h.id AS hijo_id
FROM tabla_padre p
LEFT JOIN tabla_hija h
    ON h.padre_id = p.id;
```

Debes preparar datos que incluyan al menos un caso sin coincidencia para poder observar NULL.

11. Paso 6 — Encontrar registros sin hijos

Patrón útil

```
SELECT
    p.id,
    p.nombre
FROM tabla_padre p
LEFT JOIN tabla_hija h ON h.padre_id = p.id
WHERE h.id IS NULL;
```

Adáptalo a una pregunta real. Por ejemplo: clientes sin reservas, productos sin movimientos, profesionales sin citas, equipos sin mantenimiento, etc., sólo si esa pregunta tiene sentido en tu dominio.

12. Paso 7 — ON y WHERE

- ON: expresa cómo se relacionan las filas.
- WHERE: expresa qué subconjunto quieres después de relacionarlas.
- No muevas condiciones sin pensar, especialmente en LEFT JOIN.

- Explica en palabras cada condición antes de ejecutarla.

13. Paso 8 — Comprende la cardinalidad del resultado

Una fila padre puede repetirse si tiene varios hijos. Eso no significa automáticamente que el JOIN esté mal.

Ejemplo mental

Si un cliente tiene 3 pedidos y consultas cliente + pedido, aparecerán 3 filas para ese cliente porque cada fila representa una combinación distinta cliente–pedido. No uses DISTINCT hasta entender qué representa cada fila.

14. Paso 9 — Provoca y diagnostica un error

Con autorización del docente, crea una copia temporal de una consulta y modifica deliberadamente su ON. No dañes datos. El objetivo es aprender a diagnosticar resultados incoherentes.

- Compara cantidad de filas antes/después.
- Revisa si uniste PK con la FK correcta.
- Busca crecimiento multiplicativo.
- Explica por qué “la consulta ejecutó” no significa “la consulta es correcta”.

15. Archivo obligatorio de la Clase 08

Ruta sugerida

```
backend/src/main/resources/db/queries/class08_joins.sql
```

Tu archivo debe contener como mínimo:

- Q1: INNER JOIN de dos tablas.
- Q2: JOIN de tres tablas, si tu modelo implementado lo permite.
- Q3: JOIN con filtro WHERE.
- Q4: LEFT JOIN.
- Q5: consulta de registros sin coincidencia mediante IS NULL.
- Q6: una consulta adicional importante de tu dominio.
- Cada consulta debe tener comentario con la pregunta de negocio y una breve explicación.

Formato recomendado

```
-- Q1. ¿Qué ...?
-- Relaciones: tabla_hija.fk -> tabla_padre.id
-- Cada fila representa: ...
SELECT ...;
```

16. Checklist técnico antes del commit

- ☐ Cada ON está justificado por una FK real.
- ☐ No usaste SELECT * como solución final.
- ☐ Los alias son claros.
- ☐ Puedes explicar qué representa cada fila.
- ☐ Probaste al menos un caso sin coincidencia para LEFT JOIN.
- ☐ No utilizaste DISTINCT para esconder un problema.
- ☐ Las consultas funcionan sobre el dataset de tu proyecto.
- ☐ Los comentarios describen preguntas reales del dominio.

17. Revisión cruzada obligatoria

1. Un integrante abre la consulta escrita por otro integrante.
2. Sin ejecutarla, explica qué tablas participan y qué resultado espera.
3. Localiza cada FK en el DER o DDL.
4. Ejecuta la consulta y compara con su predicción.
5. Si no puede explicarla, el equipo todavía no puede considerarla terminada.

18. Commit de la Clase 08

Ejemplo

```
git status
git add backend/src/main/resources/db/queries/class08_joins.sql
git commit -m "feat(db): add join queries for class 08"
git log --oneline -5
```

Ajusta rutas si tu repositorio utiliza una estructura acordada diferente. No inventes una estructura paralela sólo para esta clase.

19. Evidencia mínima que tu grupo debe poder mostrar

- Base de datos del proyecto con datos coherentes.
- Al menos dos relaciones identificadas mediante PK/FK.
- INNER JOIN ejecutado.
- JOIN de tres tablas cuando corresponda.
- LEFT JOIN con caso real de ausencia.
- Consulta que detecta registros sin coincidencia.
- Archivo class08_joins.sql versionado.
- Commit visible en el historial.
- Cualquier integrante puede explicar una consulta elegida por el docente.

20. Preguntas de defensa individual

- ¿Qué pregunta de negocio responde esta consulta?
- ¿Qué representa cada fila?
- ¿Cuál es la PK de esta tabla y cuál es la FK de la otra?
- ¿Por qué escribiste exactamente esa condición ON?
- ¿Qué cambia si reemplazas INNER JOIN por LEFT JOIN?
- ¿Por qué aparece NULL aquí?
- ¿Por qué se repite esta entidad en varias filas?
- ¿Qué problema tendría usar DISTINCT sin analizar primero?
- ¿Qué pasaría si eliminas la condición ON?
- Muéstrame en tu DDL la FK utilizada por este JOIN.

21. Autoevaluación rápida

Afirmación	Sí	Todavía no
Puedo encontrar una FK sin ayuda.	☐	☐
Puedo escribir un INNER JOIN desde el modelo.	☐	☐
Comprendo cuándo usar LEFT JOIN.	☐	☐
Puedo explicar ON vs WHERE.	☐	☐
Puedo explicar por qué una fila puede	☐	☐

Afirmación	Sí	Todavía no
repetirse.		
Puedo diagnosticar un JOIN incorrecto.	☐	☐
Puedo defender consultas que escribió otro integrante.	☐	☐

22. Definition of Done — tu grupo

- ☐ Las consultas usan las tablas reales de TU proyecto.
- ☐ Las relaciones coinciden con tu DER y DDL.
- ☐ Existe evidencia de INNER JOIN y LEFT JOIN.
- ☐ Cada consulta tiene intención de negocio.
- ☐ El equipo comprende la cardinalidad del resultado.
- ☐ El archivo SQL está en Git.
- ☐ No se copió el dominio de ParkFlow 360.
- ☐ Al menos dos integrantes pueden defender cualquier consulta del archivo.

23. Preparación para la Clase 09

Próximo paso

La Clase 09 trabajará agregaciones: COUNT, SUM, AVG, MIN, MAX, GROUP BY y HAVING. Antes de llegar, asegúrate de que tu dataset tenga suficiente variedad para poder calcular indicadores reales de tu proyecto.